

Resultados — Temporización No Bloqueante con millis()

Sistemas Programables — Parte 2

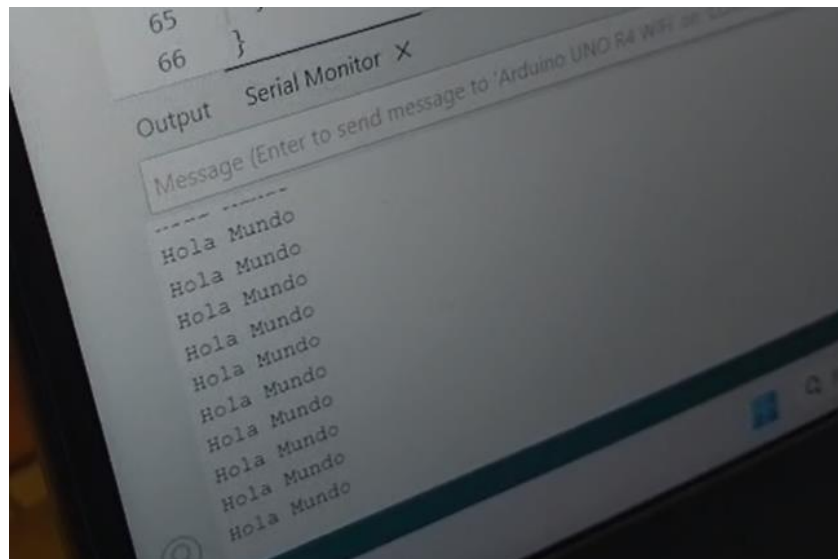
1. Comportamiento observado por tarea

A diferencia de la Parte 1, cada una de las cuatro tareas del programa (tres LEDs y el mensaje serial) se ejecutó de forma independiente, sin bloquear a las demás:

Tarea	Intervalo	Comportamiento observado	Independencia
LED1	500 ms	Parpadea de forma continua e independiente	Sin interferencia de LED2 ni LED3
LED2	1000 ms	Parpadea de forma continua e independiente	Sin interferencia de LED1 ni LED3
LED3	1500 ms	Parpadea de forma continua e independiente	Sin interferencia de LED1 ni LED2
Serial	3000 ms	Imprime "Hola Mundo" en el Monitor Serial	No afecta el parpadeo de los LEDs

2. Evidencia del Monitor Serial

Captura del Monitor Serial mostrando la impresión periódica de "Hola Mundo" cada 3 segundos, ocurriendo en paralelo mientras los tres LEDs seguían parpadeando en el circuito físico:



3. Comparación de ciclos: delay() vs. millis()

En la Parte 1, un ciclo completo del programa tardaba 6000 ms debido a la suma de todos los delay(). En esta Parte 2, no existe un "ciclo completo" en ese sentido: el loop() se ejecuta continuamente, miles de veces por segundo, y

cada tarea se dispara de forma independiente en cuanto se cumple su propio intervalo. Esto significa que LED1 efectivamente parpadea cada 500 ms de manera real, sin depender de LED2, LED3 ni de la tarea del mensaje serial.

4. Observaciones generales

- Los tres LEDs mostraron un parpadeo simultáneo, cada uno a su propia velocidad, algo que no era posible con `delay()`.
- El mensaje "Hola Mundo" se imprimió de forma consistente cada 3 segundos, sin retrasos ni saltos, incluso mientras los LEDs cambiaban de estado.
- No se observó ningún bloqueo perceptible del programa: el sistema respondió de forma fluida durante toda la prueba.
- Agregar la tarea del mensaje serial no requirió modificar la lógica existente de los LEDs, confirmando que el patrón con `millis()` es fácilmente escalable.

5. Conclusión de resultados

Los resultados confirman que la temporización basada en `millis()` logra el comportamiento originalmente buscado en la Parte 1: múltiples tareas con distintos intervalos ejecutándose de forma verdaderamente concurrente. La comparación directa entre ambas partes de la práctica deja en evidencia la superioridad de este enfoque no bloqueante para cualquier proyecto que requiera manejar más de una temporización al mismo tiempo.